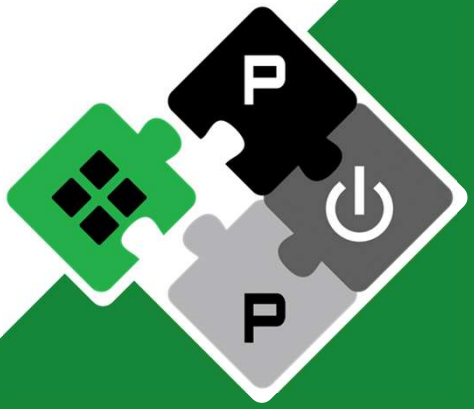
ETH zürich

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Pointwise (1×1) Convolutions on the AMD XDNA2 NPU

Integrated Systems Laboratory (ETH Zürich)

Andrea Massimo Rosetti arosetti@ethz.ch



@pulp_platform

pulp-platform.org

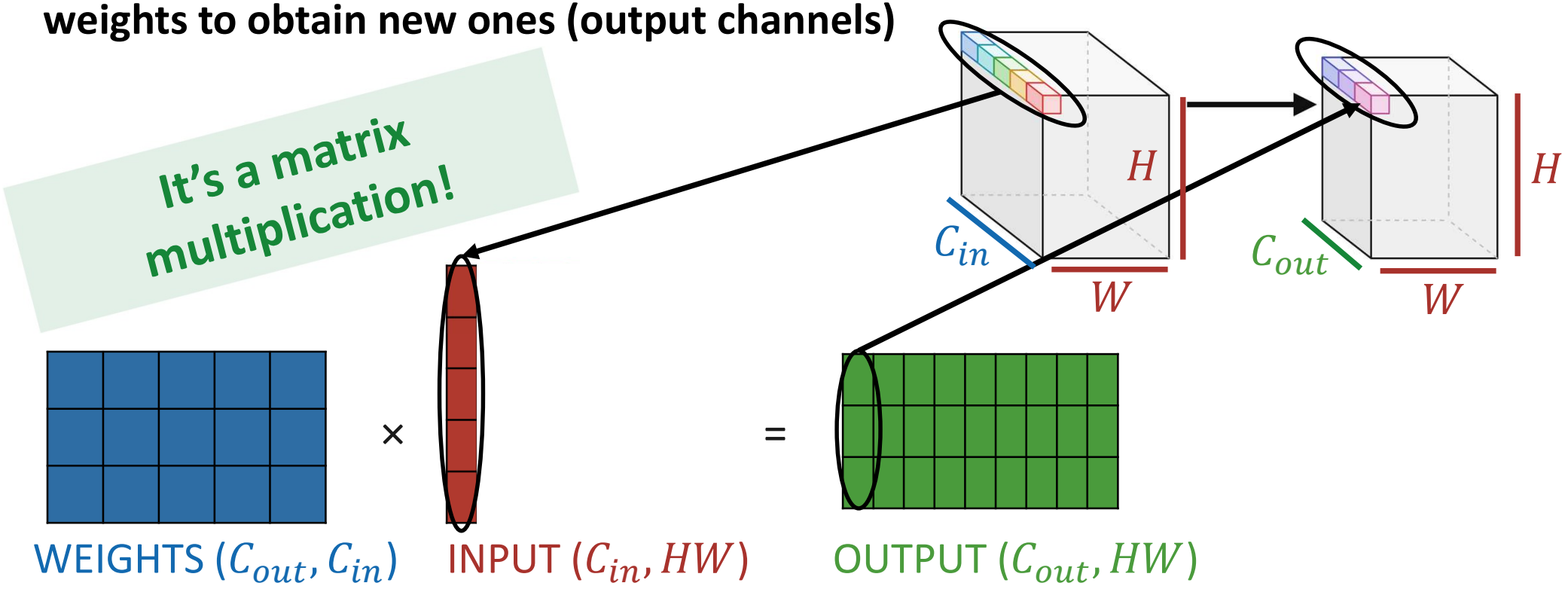
youtube.com/pulp_platform

PULP Platform

Open Source Hardware, the way it should be!

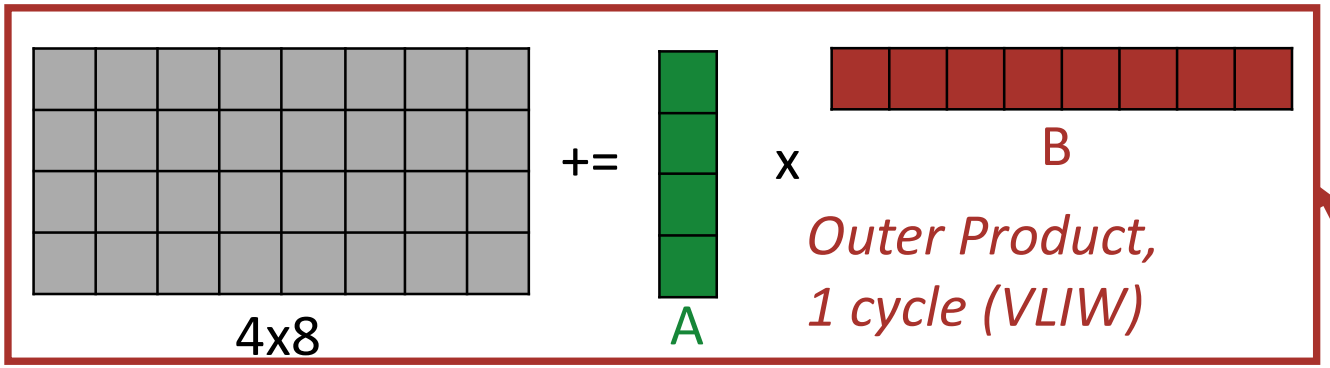
Pointwise Convolution as Matrix Multiplication

- **Every pixel carries a stack of features (input channels)**
- **A Pointwise (or 1x1) Convolution combines these features with some fixed weights to obtain new ones (output channels)**

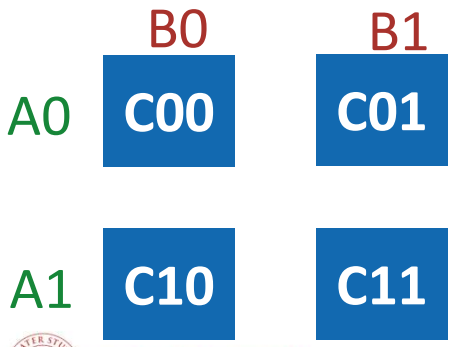


What each core does (GEMM)

- **A grid of 32 cores interconnected with a NoC**
- **Each core computes one small tile of the output layer**

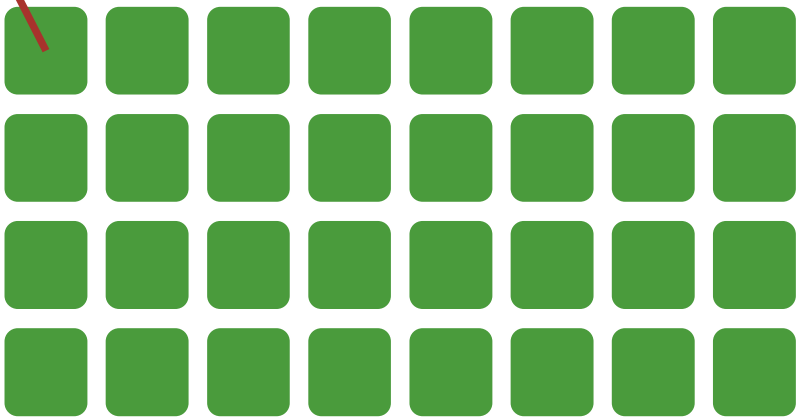


2x2 accumulators, each operand used twice



4 rows

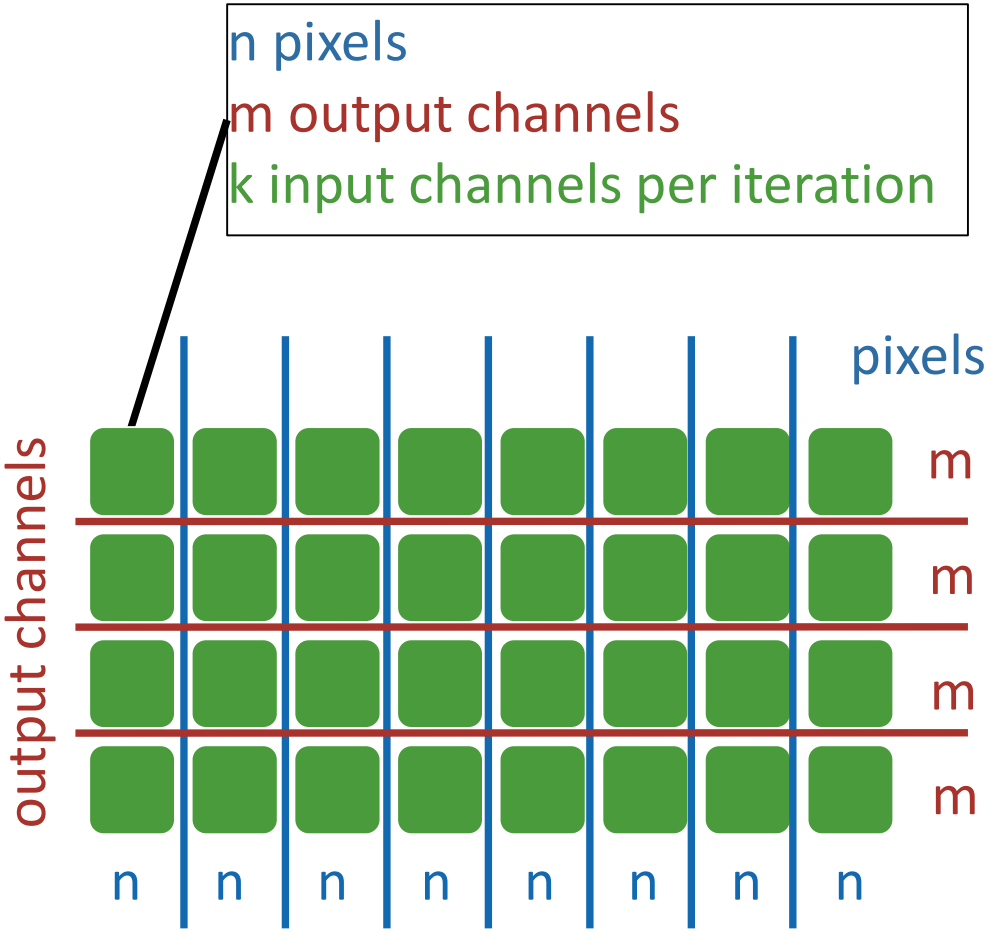
8 columns



Splitting the multiplication across the grid (GEMM)

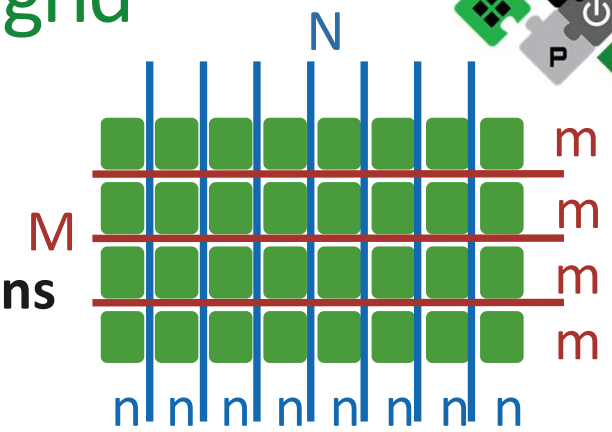
- **There are three sizes to split**
  - M: output channels
  - N: pixels (N = HW)
  - K: input channels (hidden size)

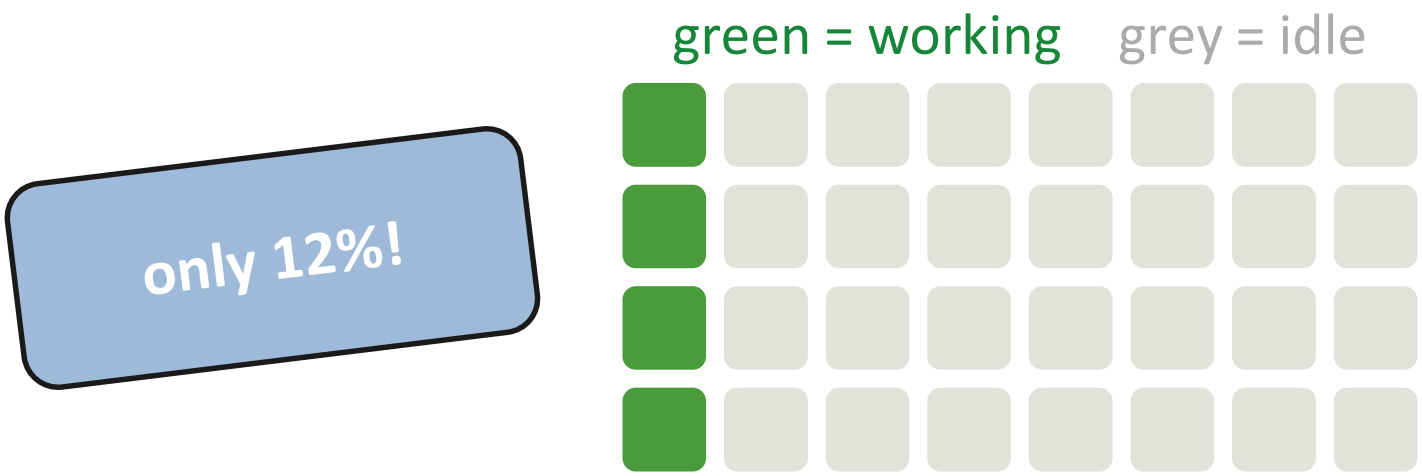
- **How to choose n, m, k?**
  - must fit inside the 64KiB L1 scratchpad
  - Trade-off: higher values mean larger data reuse, but if inputs are small we waste computation
  - $n = 64, k = 32, m = 16$



The weakness: tall layers don't use the entire grid

- **In many cases: $M \gg N$ (tall layers)**
  - Example from ConvNeXt-Tiny : 3072 channels, only 49 pixels (< n = 64).
- **Columns handle pixels but 49 pixels fill only 1 of the 8 columns**
- **The other 7 columns compute zeros**

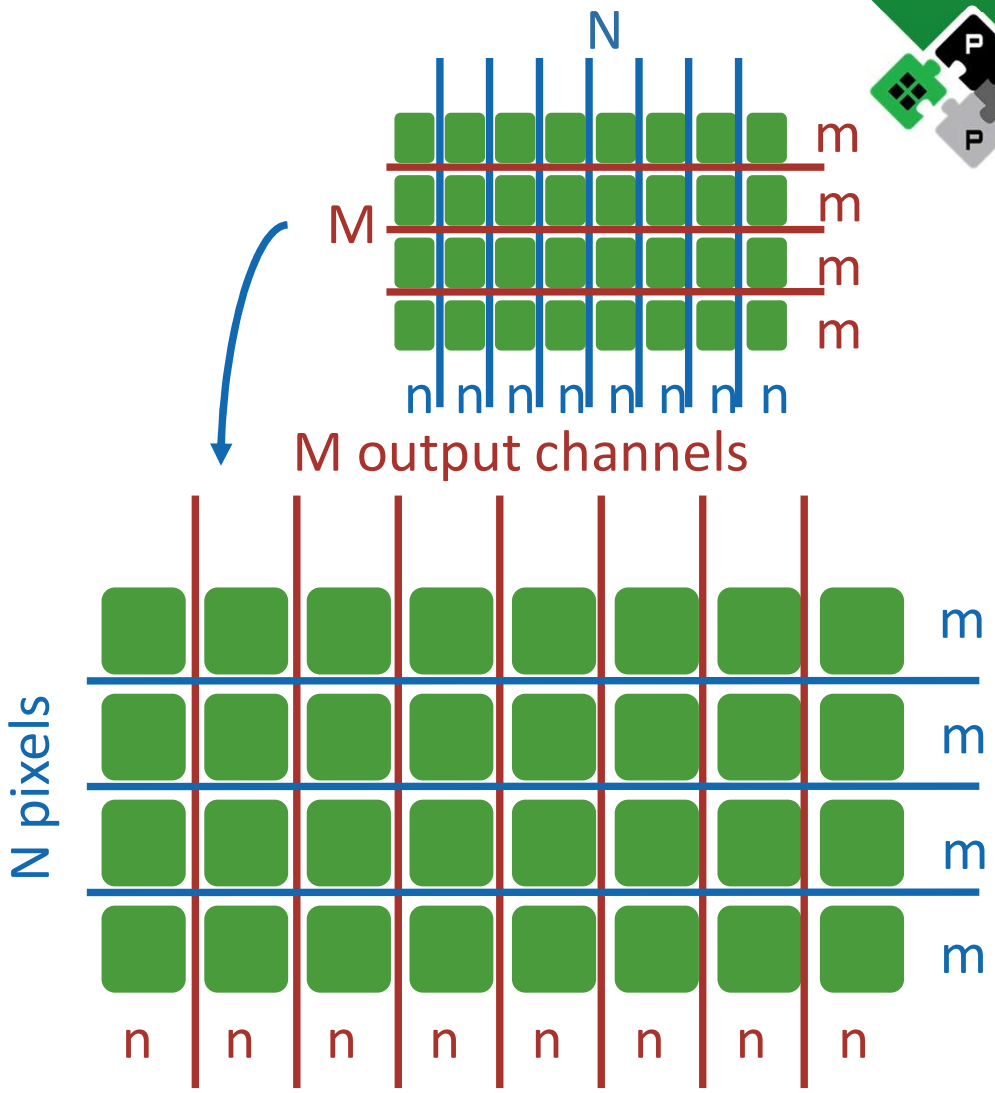




1 column × 4 rows = **4 of 32 workers** = **12%** of the chip.

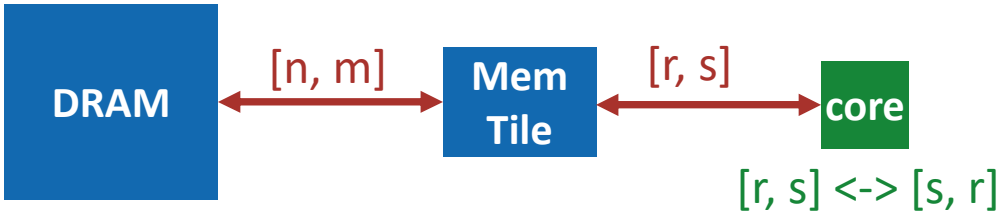
Swapping the Axes

- **For tall layers, we invert the assignment**
  - Columns occupied: $\max\left(8, \frac{3072}{n}\right) = 8$, with n = 64
  - Rows occupied: $\max\left(4, \frac{49}{m}\right) = 4$, with m = 16
- **The cost**
  - Now m = 16 is the tile size on the contiguous axis, so the DMA burst length for pixels is reduced by 4



Swap Implementation

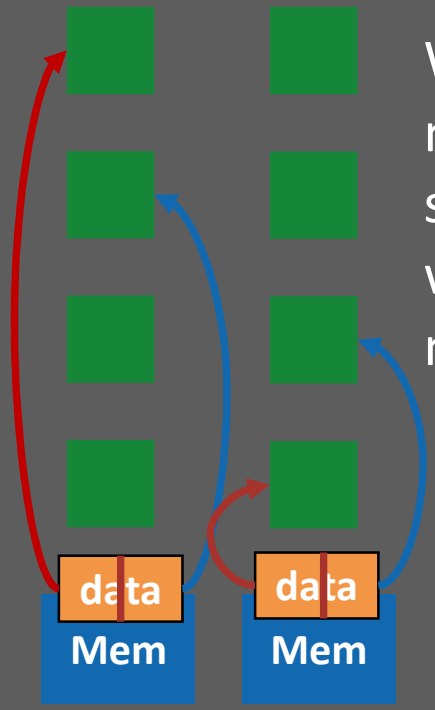
- **Property of matrix multiplication:** $\boldsymbol{AB = (B^T A^T)^T}$
- **We need to transpose A, B and C.**
- **Transpose happens in the core**
  - aie::transpose is easy to use
  - r, s are the sizes the core operates on




```
if M > N and (N/n)*2 <= n_col:
    a_col_maj = 1
```
new flag

```
    b_col_maj = 1
    c_col_maj = 1
```
already implemented in GEMM

Matrix A is row-distributed



We only swap when n_col >= 4, to avoid splitting data from A when it is in column major order.

Methodology

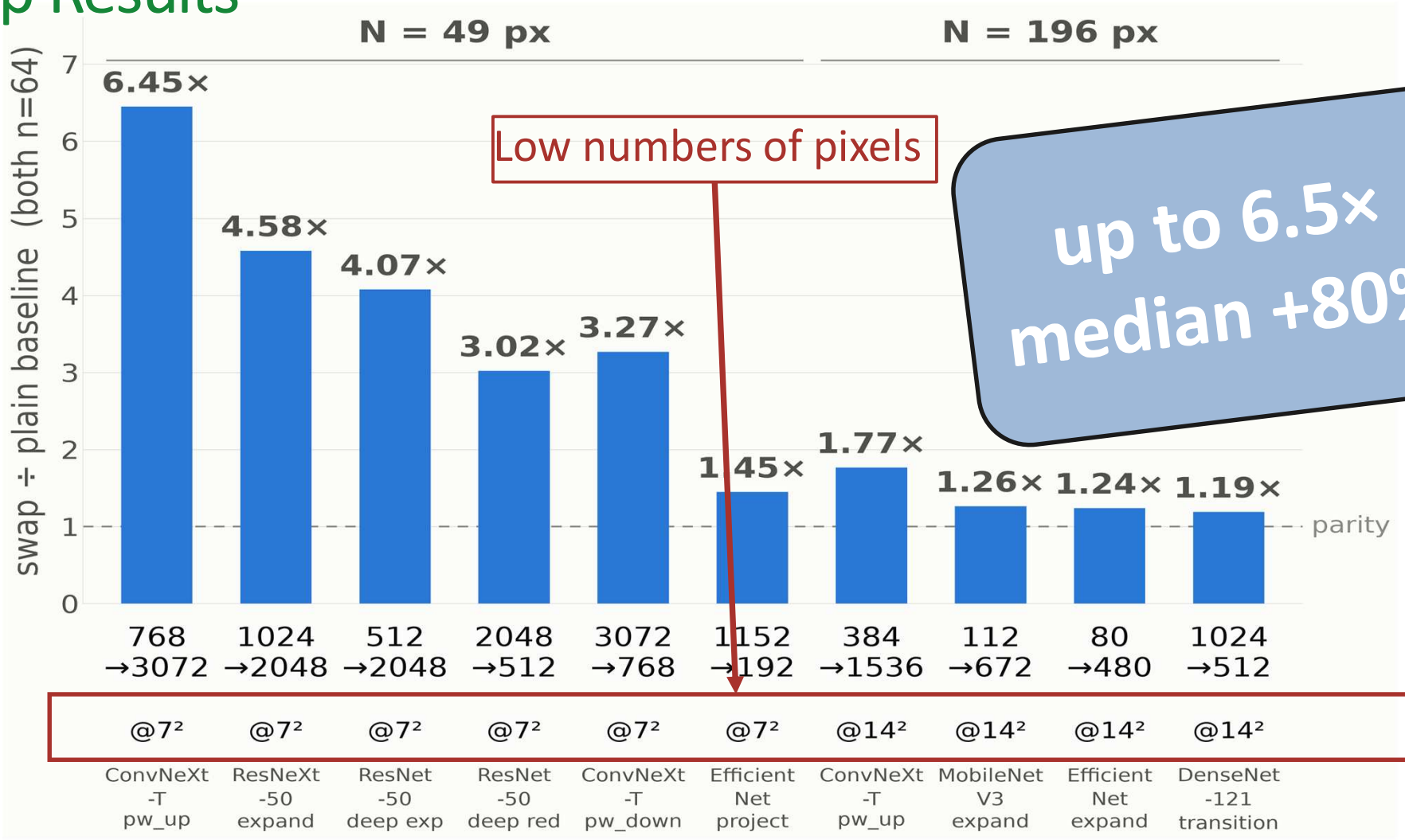
10 warmup rounds, 50 iterations

Check correctness against Pytorch golden model

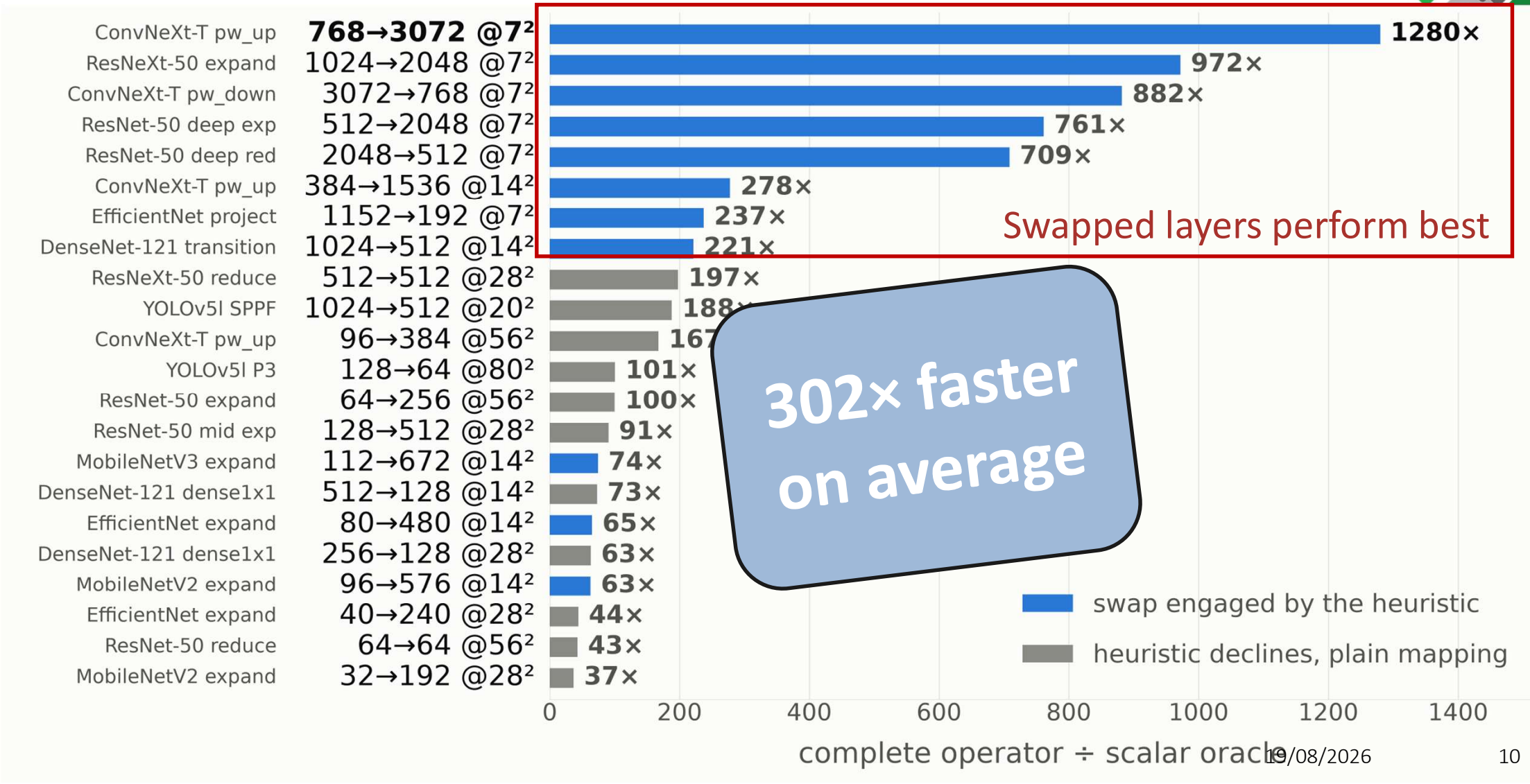
$$Time = npu\_time\ (\text{median})$$

$$\text{Throughput} = \frac{useful\_flops}{npu\_time}$$

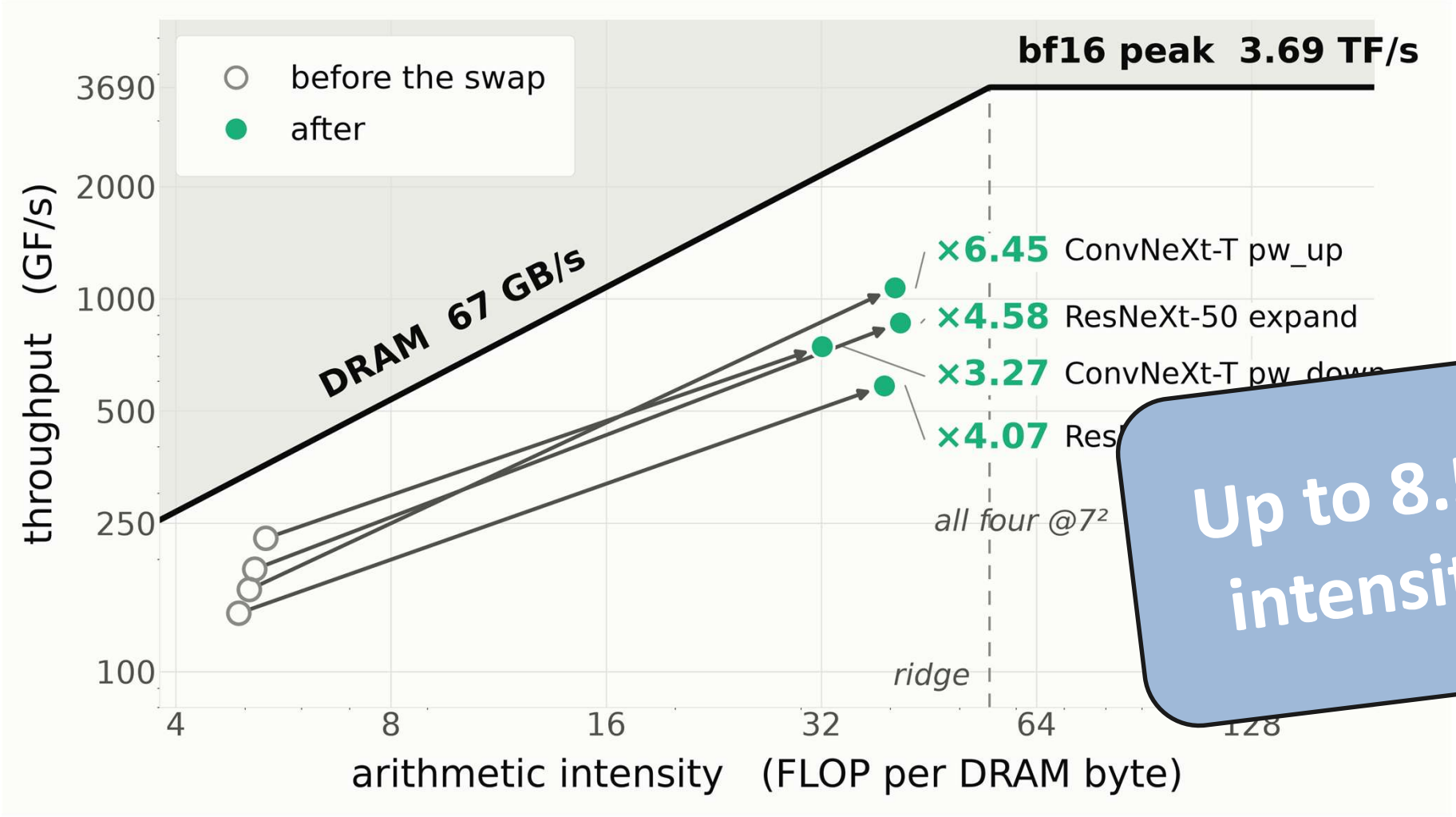
Swap Results



Parallel vs naive Scalar kernel



Roofline Plot before and after swap



Conclusions

- **The engine is slowest when it is empty**
  - Scalar kernel cannot keep up feeding the entire grid
  - Plain mapping with tall layers starves columns

302x mean against simple scalar kernel

Up to 6.45x with a simple mapping change

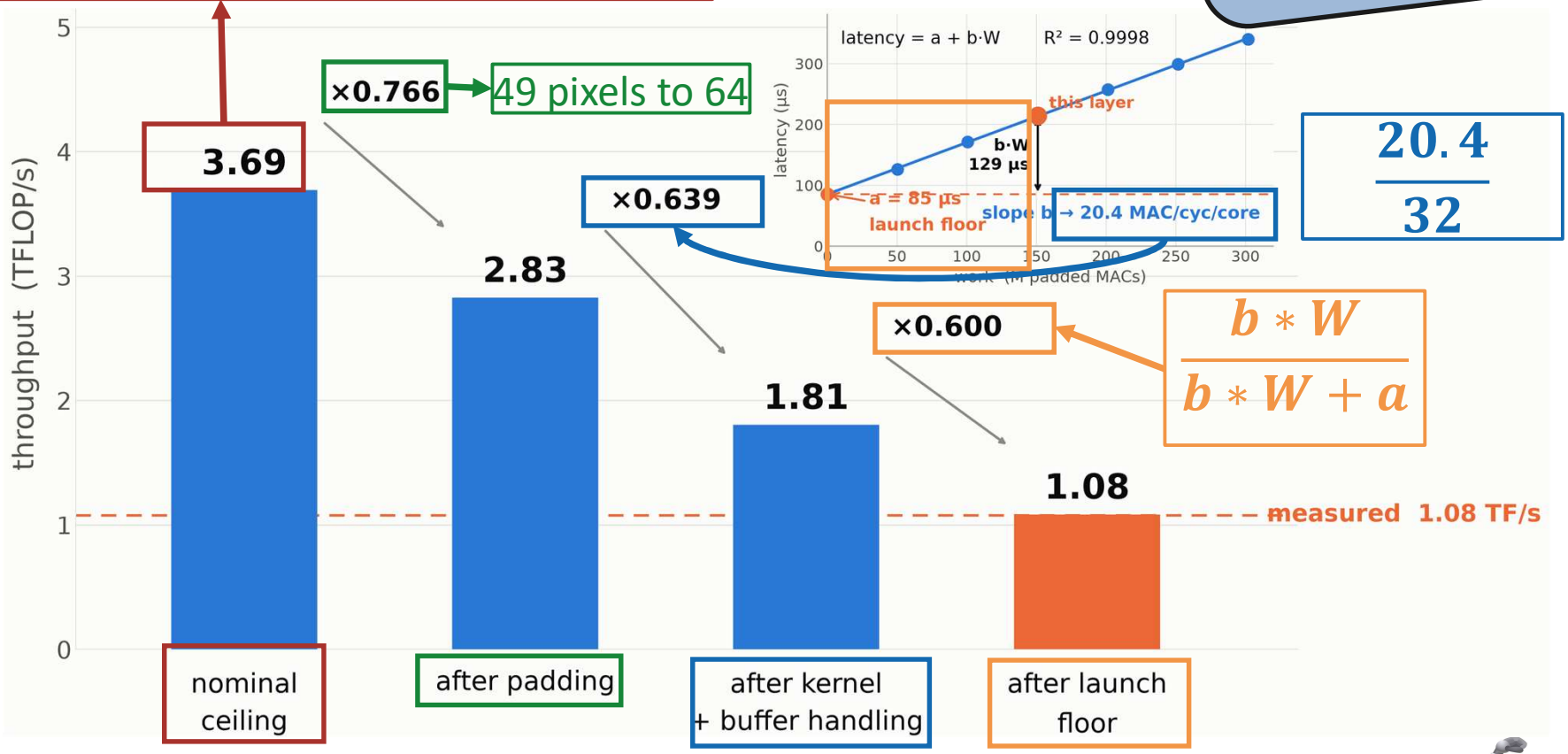
- **Limits and Future Work**
  - Intrinsic overheads (setup, kernel…)
  - Padding due to input shape
  - Profiling to pinpoint efficiency limits

3-36% of theoretical throughput reached

Cycles Budget (ConvNeXt-T, 768->3072, 49 px)

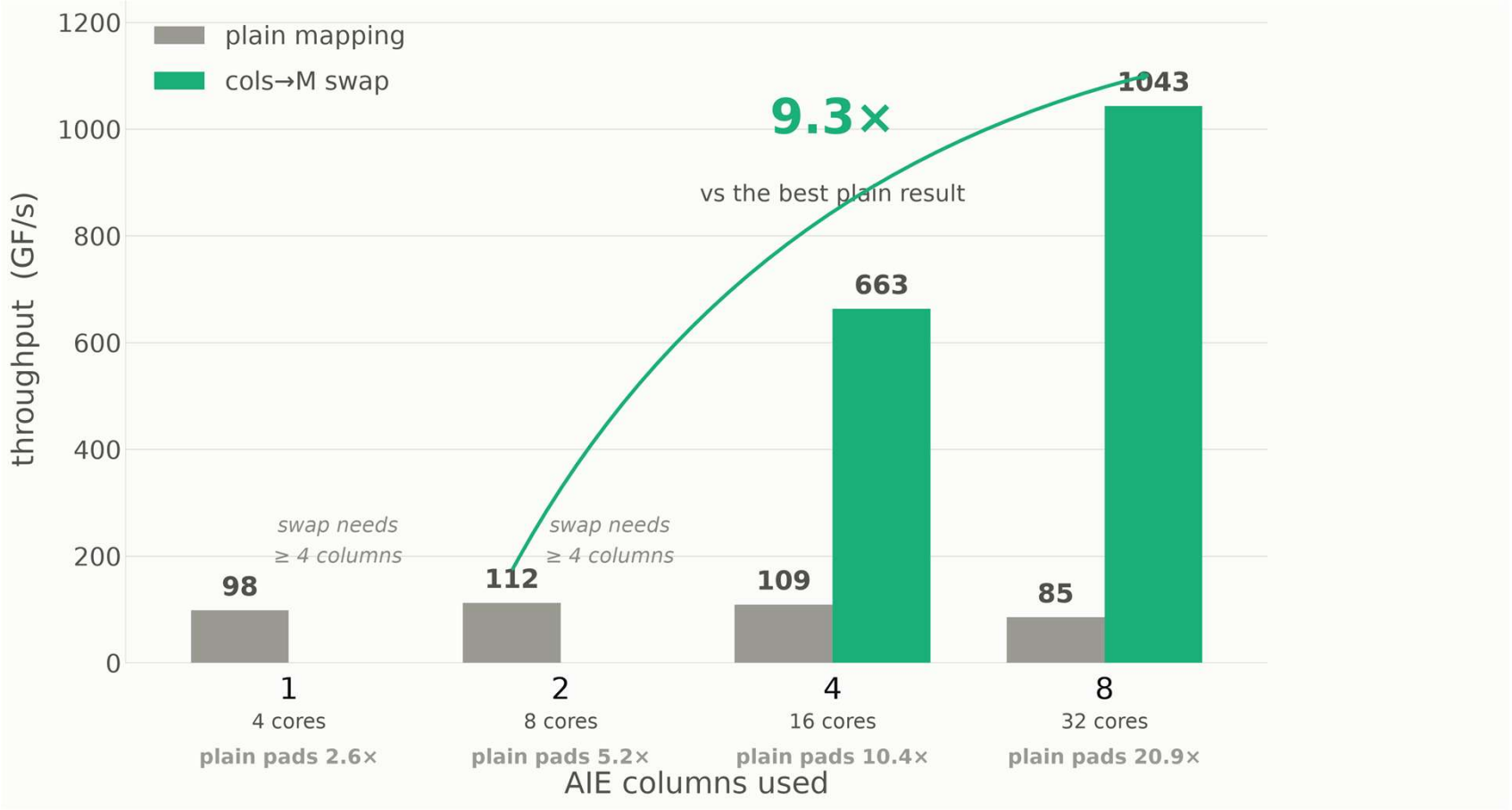
Theoretical Ceiling: 256 MACs per logical mmul,
but bf16 is emulated 8:1, so 32 MAC/instr @ 1.8GHz

29% measured throughput accounted for!



Swap Results: Scaling with the Chip

- **As expected, swap yields better results when we use more columns of the grid.**



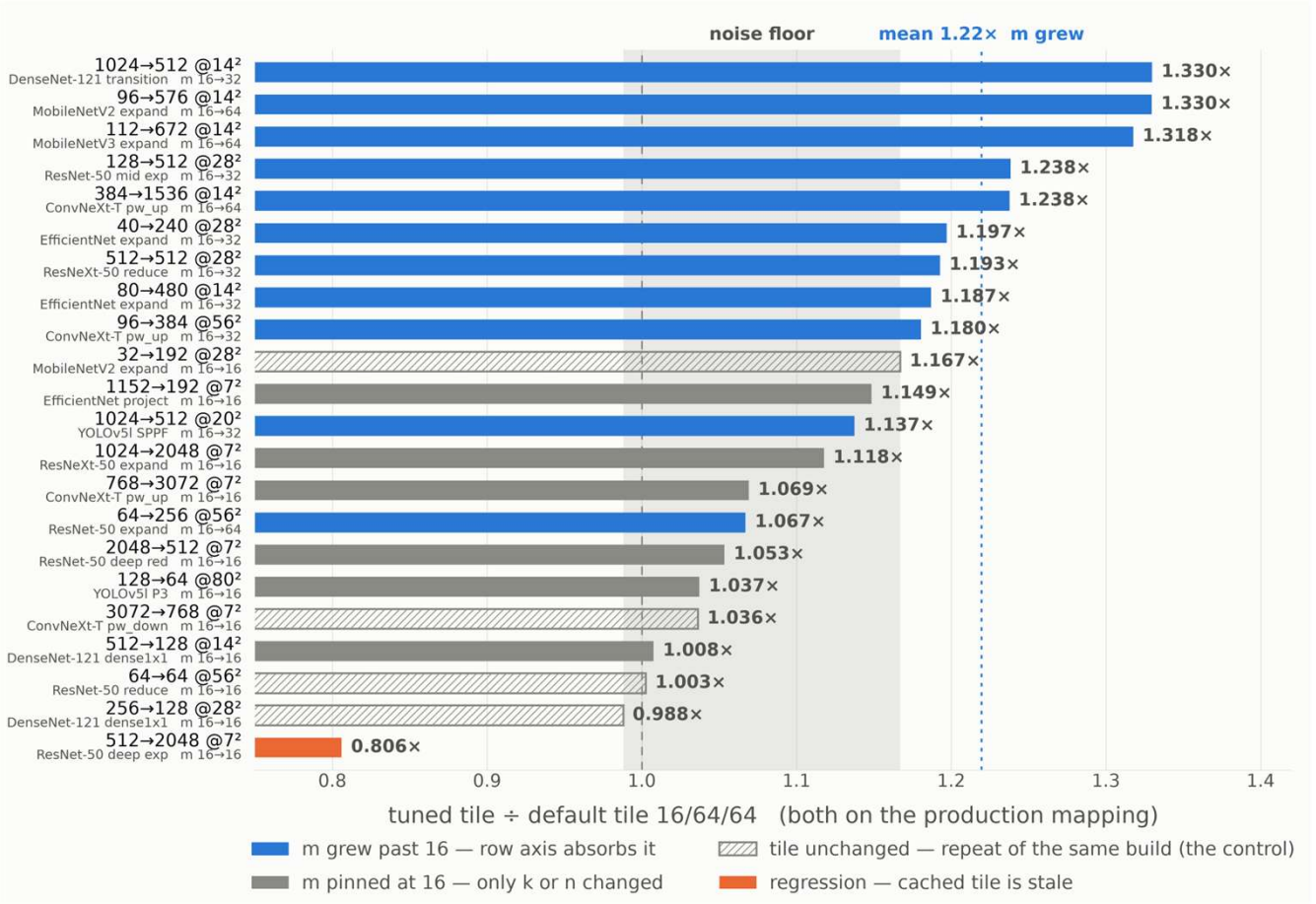
Discarded option: bfp16 (block float)

- **Idea: we don’t need very high accuracy for convolutions, let’s use something we don’t need to emulate**
- **But computation was never the bottleneck for convolutions: even 8x in datapath does not change the situation much (Amdahl’s Law), max 1.45x improvement**
- **Tests showed it costs 5 orders of magnitude in accuracy**
- **Incompatible with swap (not supported by aie::transpose), which is a better option**

A minor gain: fine tuning tile size

- Each layer also has **tile settings configuring** how the work is chopped up.
- We **try a few on the real chip** and keep the fastest.

1.33x max, not a huge gain



Swap vs finer column tile